fitosim

Libreria Python per la simulazione agronomica di piante in vaso

Stato del progetto al 26 aprile 2026

1. Panoramica

fitosim è una libreria Python che implementa il metodo FAO-56 di Allen, Pereira, Raes e Smith (1998) per la stima dell'evapotraspirazione di riferimento e per il bilancio idrico di colture in vaso, applicato al contesto del giardinaggio domestico. Nasce come fondamenta algoritmica per il progetto «Il Mio Giardino» dell'autore (Andrea, basato a Milano) e come piattaforma per esperimenti di calibrazione del modello con letture dirette di sensori reali.

Il progetto è stato costruito incrementalmente attraverso una serie di tappe didattiche: ogni tappa ha introdotto un concetto fisico o agronomico, lo ha tradotto in codice, lo ha validato con test e lo ha illustrato con esempi visualizzati. Lo stile di sviluppo è quello di una piccola libreria scientifica: dipendenze ridotte all'osso (motore in stdlib pura, matplotlib usato solo in test ed esempi), architettura a tre livelli ben separati, copertura test estesa, ogni formula esplicitamente collegata al riferimento FAO-56 di provenienza.

Numeri di sintesi

Linguaggio	Python ≥ 3.10 (uso di str | None, dataclass, ecc.)
Dipendenze runtime	Solo standard library (urllib, json, sqlite3, dataclasses)
Dipendenze test/esempi	matplotlib, numpy, pytest
Architettura	Tre livelli (science / domain / io) con dipendenze unidirezionali verso il basso
Moduli sorgente	9 moduli (4 science, 3 domain, 2 io); ~3 700 righe di codice
Test	220 test verdi su 9 file di test; ~3 100 righe di test
Esempi dimostrativi	15+ script eseguibili con grafici matplotlib
Lingua del codice	Italiano (docstring, commenti, output testuali)

2. Architettura a tre livelli

L'architettura è organizzata in tre livelli concentrici, ciascuno con un proprio ruolo epistemico ben definito. Il principio fondamentale è che le dipendenze fluiscono soltanto verso il basso: il livello scientifico non sa nulla del dominio biologico né delle API esterne; il livello dominio consuma il livello scientifico ma non sa nulla di Open-Meteo o di Ecowitt; il livello io collega il

dominio al mondo esterno ma non viene mai importato dal dominio. Questa stratificazione è ciò che rende fitosim facile da estendere e refactorare senza paura di rotture.

science/ — fisica generale

Il livello scientifico contiene la matematica universale del fenomeno evapotraspirativo: equazioni che valgono allo stesso modo a Milano e a Tokyo, per un basilico o per un olivo. Sono funzioni pure, in stdlib, indipendenti da qualunque concetto biologico specifico.

- radiation.py — radiazione extra-atmosferica R_a (FAO-56 eqs. 21-25)
- et0.py — Hargreaves-Samani per ET_0 a partire dalla sola escursione termica
- substrate.py — proprietà idrauliche dei substrati: TAW, RAW, conversioni θ↔mm
- balance.py — bilancio idrico di un singolo passo + coefficiente di stress Ks (eq. 84)

domain/ — biologia specifica

Il livello dominio cattura le entità biologiche e operative: questa specie consuma più di quella, questo vaso ha un substrato e una geometria specifici, questo piano di irrigazione consiglia interventi su date precise. È il livello che dà significato agronomico ai numeri prodotti dal livello scientifico.

- species.py — Species + catalogo di 5 specie (basilico, pomodoro, lattuga, limone, rosmarino), modello fenologico time-driven
- pot.py — Pot come entità unificata che lega specie + substrato + geometria + stato + posizione + data di impianto
- scheduler.py — pianificatore di irrigazioni via simulazione forward; distingue allerte già attive da allerte previste

io/ — contatto col mondo esterno

Il livello io porta dentro fitosim i dati reali del mondo: previsioni grigliate da Open-Meteo, letture dirette dei sensori della stazione Ecowitt installata sul balcone. Il pattern uniforme è: parser puro (testabile con dizionari fissi) + funzione di fetch con fetcher iniettabile (testabile senza chiamate di rete).

- openmeteo.py — client per previsione e archivio storico, con cache su disco e fallback offline
- ecowitt.py — client per real_time e history della stazione, con conversioni unità imperial→metric e aggregazione sub-oraria→giornaliera

3. Mappa dei moduli

Quella che segue è una vista di alto livello di ciascun modulo con le sue responsabilità, le entità principali esposte e i riferimenti scientifici che lo motivano.

science/radiation.py (194 righe)

Radiazione solare extra-atmosferica R_a, ovvero la radiazione che arriverebbe alla superficie in assenza di atmosfera. È l'input astronomico al motore Hargreaves-Samani.

- day_of_year(d) — giorno juliano nell'anno
- inverse_relative_distance(j), solar_declination(j), sunset_hour_angle(lat, decl) — pezzi geometrici di R_a
- extraterrestrial_radiation(lat, j) → R_a in MJ/m²/giorno

Validato contro FAO-56 Esempio 8 (lat=-20°, J=246 → R_a=32.2).

science/et0.py (150 righe)

Evapotraspirazione di riferimento ET_0 secondo Hargreaves-Samani: formula approssimata che richiede solo T_min, T_max e R_a. Compromesso voluto: meno preciso di Penman-Monteith ma usa solo le grandezze più facilmente misurabili in un balcone domestico.

- et0_hargreaves_samani(t_min, t_max, latitude_deg, j) → mm/giorno
- Costanti di conversione: LATENT_HEAT_VAPORIZATION=2.45 MJ/kg

Validato vs Penman-Monteith di Open-Meteo: bias medio ~5% nella stagione di crescita (vedi figura openmeteo_et0_validation.png).

science/substrate.py (385 righe)

Modellizzazione delle proprietà idrauliche del substrato: capacità di campo θ_FC, punto di appassimento θ_PWP, total available water TAW, readily available water RAW.

- Substrate (frozen dataclass) — un substrato tipico ha θ_FC e θ_PWP come parametri primari
- Catalogo: PEAT_BASED, COCO_COIR, UNIVERSAL_POTTING_SOIL, PERLITE_RICH, CACTUS_MIX
- Funzioni geometriche: circular_pot_surface_area_m2, pot_substrate_depth_mm
- Conversioni: theta_to_mm, mm_to_theta — fanno da bridge tra unità FAO (θ adimensionale) e ingegneria (mm di colonna)

science/balance.py (346 righe)

Cuore del motore: bilancio idrico di un singolo giorno e coefficiente di stress Ks. Implementazione FAO-56 eq. 84 lineare a tratti.

- water_balance_step — funzione unit-agnostic generica
- water_balance_step_theta, water_balance_step_mm — wrapper specifici per le due unità
- BalanceStepResult — dataclass con new_state, drainage, under_alert
- stress_coefficient_ks(θ, substrate, p) — Ks ∈ [0,1] modula ET_c nella zona di stress

domain/species.py (357 righe)

Specie coltivabile come entità immutabile con coefficienti colturali e modello fenologico.

- PhenologicalStage (Enum): INITIAL, MID_SEASON, LATE_SEASON
- Species (frozen dataclass) — Kc trio + depletion_fraction + initial_stage_days + mid_stage_days
- species.stage_at_day(days_since_planting) — calcolo dinamico dello stadio

- kc_for_stage, potential_et_c, actual_et_c — composizione Kc × ET_0 × Ks
- Catalogo: BASIL, TOMATO, LETTUCE, CITRUS, ROSEMARY

domain/pot.py (288 righe)

Vaso fisico unificato: una specie + un substrato + una geometria + uno stato + una data di impianto. È il punto in cui l'ergonomia del codice cliente fa il salto di qualità.

- Location (Enum): OUTDOOR, INDOOR — discrimina la fonte meteo da usare
- Pot — l'unico campo mutabile è state_mm; tutto il resto è immutabile
- Properties derivate: surface_area_m2, substrate_depth_mm, fc_mm, pwp_mm, alert_mm
- apply_balance_step(et_0, water_input, current_date) — un giorno di simulazione su questo vaso
- water_to_field_capacity_liters() — la dose di irrigazione raccomandata in litri

domain/scheduler.py (336 righe)

Pianificatore agronomico: data inventario + previsione, produce un piano di interventi consigliati con date e dosi. Lavora su copie dei vasi via dataclasses.replace per non inquinare lo stato reale.

- IrrigationReason (Enum): CURRENTLY_IN_ALERT, PREDICTED_ALERT
- IrrigationEvent — singolo intervento con data, vaso, dose_mm, dose_liters, ragione
- IrrigationPlan — collezione + helper di consultazione (events_for_pot, days_with_events, total_liters_on_date, ecc.)
- plan_irrigations(inventory, forecast, latitude_deg, today) — funzione principale

io/openmeteo.py (539 righe)

Client per la API Open-Meteo: previsione fino a 16 giorni e archivio storico dal 1940. ET_0 FAO Penman-Monteith fornito direttamente dal servizio per validazione esterna.

- DailyWeather (frozen dataclass) — formato canonico consumato dal motore
- fetch_daily_forecast — con cache su disco (TTL 6h), fallback a cache scaduta su errore di rete, fetcher iniettabile
- fetch_daily_archive — endpoint storico con date espliciti
- parse_openmeteo_response — parser puro, separato dal fetching

io/ecowitt.py (1102 righe — il modulo più ricco)

Client per la stazione Ecowitt domestica. Tre endpoint, due dataclass principali, conversione completa imperial→metric, aggregazione temporale.

- EcowittObservation — snapshot real-time con outdoor, indoor, WN31 multi-canale, WH51 multi-canale, vento, pressione, pioggia
- EcowittTimeSeries + EcowittSeriesPoint — serie storica sub-oraria
- Conversioni: _to_celsius, _to_mm, _to_mm_per_hour, _to_m_per_second, _to_hpa con gestione dei due caratteri di grado
- fetch_real_time, fetch_history — entrambe con fetcher iniettabile

- aggregate_to_daily_weather — il bridge: trasforma serie sub-orarie in DailyWeather identico al formato Open-Meteo
- credentials_from_env — lettura sicura delle credenziali da variabili d'ambiente

4. Esempi dimostrativi

La directory examples/ contiene gli script che illustrano l'uso della libreria. Ogni esempio è eseguibile autonomamente, produce output testuale informativo a terminale e (quasi sempre) salva uno o più grafici in output/plots/. Sono ordinati grossomodo per ordine di crescente complessità: i primi mostrano funzionalità isolate del livello scientifico, gli ultimi sono demo end-to-end che integrano più livelli.

Script	Cosa illustra
milan_monthly_radiation.py	Tabella di R_a mensile per Milano — primo demo del livello science
milan_monthly_et0.py	ET_0 Hargreaves-Samani mensile su valori climatologici
substrate_comparison.py	Cinque substrati a confronto su un vaso da 5L: TAW, RAW, soglia di allerta
plots_validation.py	R_a multi-latitudine, ET_0 Milano vs altre città, heatmap globale
vaso_milano_14giorni.py	Prima simulazione end-to-end di 14 giorni — duplicato in θ e mm
plots_balance_simulation.py	Timeline + stacked bar dei livelli idrici di una simulazione
basil_with_stress_coefficient.py	Confronto traiettoria con e senza Ks: dimostra il superamento del plateau terminale
multispecies_comparison.py	Cinque specie sullo stesso vaso/meteo: traiettorie + dinamica di Ks
inventory_status_report.py	Report di stato di un inventario di 4 vasi dopo 30 giorni; ordinato per urgenza
openmeteo_milan_demo.py	Previsione reale Milano + validazione HS vs PM + simulazione inventario
scheduler_demo.py	Pianificazione settimanale: agenda testuale + Gantt + traiettorie con marker eventi
ecowitt_dashboard.py	Dashboard 4-quadranti della stazione live (con fallback a fixture)
history_bridge_demo.py	Confronto ET_0 HS calcolato da Ecowitt history vs Open-Meteo archive

Nota di pulizia: gli script multi_species_comparison.py e openmeteo_real_forecast.py sono residui di iterazioni precedenti, duplicati di multispecies_comparison.py e openmeteo_milan_demo.py rispettivamente. Possono essere eliminati a piacere senza perdita di funzionalità.

5. Decisioni di design chiave

Le scelte progettuali che hanno guidato il progetto, raccolte qui per memoria. Quasi tutte sono state validate da test specifici, alcune sono scelte di stile che non hanno test ma sono coerentemente applicate.

Tre livelli con dipendenze unidirezionali

science → domain → io. Mai il contrario. Questo ha permesso, per esempio, di scrivere e testare l'intero motore scientifico (oltre 70 test) prima di qualunque modulo io, e di rimpiazzare uno qualsiasi dei moduli io senza toccare il resto. È la scelta che paga di più sul lungo periodo.

Dual API θ ↔ mm con motore unit-agnostic

water_balance_step è una funzione pura che lavora su numeri astratti; i wrapper water_balance_step_theta e water_balance_step_mm aggiungono il contesto delle unità. Risultato: il motore non ha bug di conversione perché non sa nemmeno cosa siano le unità; le conversioni vivono ai bordi.

Pot mutabile solo in state_mm

Tutti gli altri campi del Pot sono effettivamente immutabili (Substrate e Species sono frozen dataclass). Solo state_mm cambia, ed è esattamente la grandezza che fisicamente cambia nel tempo. apply_balance_step muta in-place e restituisce il risultato, segnalando esplicitamente il side-effect.

Stadio fenologico calcolato dinamicamente

Il Pot non memorizza lo stadio come campo, lo calcola al volo dalla data di impianto via species.stage_at_day(). Questo evita il problema dei campi ridondanti che divergono dal sistema, e fa evolvere lo stato biologico in modo naturale col passare del tempo.

Scheduler lavora su copie

plan_irrigations crea copie dei vasi via dataclasses.replace e simula su quelle. Lo stato reale dell'inventario non viene mai toccato dal pianificatore, che è un consulente non un attore. Garantito da test esplicito (TestPlanIrrigationsPurity).

Parser puro + fetcher iniettabile

Tutti i moduli io seguono lo stesso pattern: parse_*_response prende un dict (testabile con dizionari letterali), fetch_* fa il GET HTTP e delega il parsing. Il fetch_* accetta un parametro fetcher opzionale per i test end-to-end senza chiamate di rete reali. Risultato: zero test che fanno rete, suite veloce e affidabile.

Cache su disco per Open-Meteo, no cache per Ecowitt history

Le previsioni Open-Meteo sono stabili a 6h e potenzialmente costose in rate limit se ripetute: cache locale a TTL 6h con fallback a cache scaduta in caso di errore di rete. Ecowitt history

invece è tipicamente interrogato una volta sola per uno specifico back-test (i dati storici non cambiano), quindi nessuna cache infrastruttura.

Real-time vs history Ecowitt: ruoli diversi

real_time fornisce snapshot istantanei dei sensori, primario per la lettura dello stato del vaso (WH51) e per il microclima indoor (WN31). history fornisce serie sub-orarie aggregabili a daily, l'unico modo per calcolare ET_0 dai dati locali (servono T_min e T_max del giorno). I due endpoint sono complementari, non sostituibili.

Aggregazione daily con soglia min_points_per_day

aggregate_to_daily_weather richiede almeno N punti per produrre una entry giornaliera. Default 4 con cadenza 30 min copre 2 ore — molto permissivo, ma protegge dal caso di una stazione installata a metà giornata che produrrebbe T_min=T_max ricavati da pochissimi sample.

6. Stato dei test

La suite è composta da 220 test (più 57 sotto-test parametrizzati). Esecuzione completa in ~1 secondo. Tutti verdi al momento della scrittura di questo documento. Strategia di test stratificata su tre livelli: validazione contro valori canonici della letteratura (es. FAO-56 Esempio 8, punti di congelamento ed ebollizione), test di robustezza su input malformati o incompleti, test di equivalenza tra cammini di codice (es. Pot vs chiamate manuali producono lo stesso risultato).

File	Test	Copertura principale
test_radiation.py	10	FAO-56 Esempio 8, monotonia, simmetrie
test_et0.py	10	Punti di rifermimento HS, range fisici
test_substrate.py	31	Catalogo, TAW/RAW, conversioni geometriche
test_balance.py	17	Bilancio idrico in θ e mm, Ks lineare a tratti
test_species.py	24	Validazione campi, fenologia, integrità catalogo
test_pot.py	20	Equivalenza Pot vs flusso manuale, fenologia da data
test_scheduler.py	22	Plan, IrrigationEvent, helper di IrrigationPlan
test_openmeteo.py	31	Parsing, cache, fetcher injection, archivio, ET_0 FAO

test_ecowitt.py	55	Conversioni unità, fixture reale, history, aggregazione
Totale	**220**	

Per rilanciare la suite:

```
$ cd fitosim/
$ python -m pytest tests/
```

7. Cosa potrebbe esserci dopo

Lo stato attuale è funzionalmente completo: la pipeline va dai dati di input (Open-Meteo o Ecowitt) attraverso il motore scientifico, fino al piano operativo di irrigazioni. Ci sono però naturali estensioni che trasformerebbero fitosim da libreria scientifica a strumento operativo quotidiano. Le elenco in ordine di valore aggiunto crescente, ciascuna con un'idea della scala di lavoro.

Calibrazione modello-vs-sensore (medio)

Aggiungere a Pot un campo wh51_channel opzionale e una funzione calibrate_from_sensor(reading_pct) che allinea state_mm alla lettura reale del sensore. In una prima versione semplice è solo un overwrite; in una versione più sofisticata può tenere traccia del delta predizione-vs-realtà nel tempo per identificare sistematicamente i vasi con θ_FC o θ_PWP malcalibrati. Questo è il vero salto di qualità: un modello che impara dai propri errori.

Estensione del motore a Penman-Monteith FAO-56 (grande)

Hargreaves-Samani è un'approssimazione; Penman-Monteith completo richiede umidità relativa, vento e radiazione netta — tutte grandezze che la stazione Ecowitt misura. Una nuova et0_penman_monteith() in science/et0.py che li consuma chiuderebbe il gap del 5% di bias che abbiamo misurato nella validazione esterna. Sarebbe un'opera di tre o quattro tappe da sole: nuova formula, nuova validazione, integrazione con Ecowitt history (che fornisce le grandezze) come fonte preferita.

Persistenza dello stato dell'inventario (piccola)

Oggi gli oggetti Pot vivono solo in memoria. Una piccola tappa per aggiungere fitosim/io/storage.py che serializza/deserializza un inventario in JSON o SQLite renderebbe possibile il workflow reale: lancia il pianificatore al mattino, applica gli interventi, salva il nuovo stato, ricarica domani. È un pezzo che probabilmente vivrà più naturalmente nel progetto «Il Mio Giardino» che ti userà come libreria, ma può anche essere un'utility di fitosim stesso.

Strato di interfaccia: CLI o app web (medio)

Una piccola CLI con typer o argparse per visualizzare l'agenda settimanale al terminale, oppure un'app web Flask/FastAPI che renderizzi il Gantt e le traiettorie del pianificatore nel browser. La seconda è più ambiziosa ma è il primo passo verso un compagno di giardinaggio vero, integrabile con notifiche Termux come nel progetto «Il Mio Giardino».

Sensori WH52 e altre estensioni hardware (zero lavoro)

Quando arriveranno i WH52, il parser ecowitt.py li raccoglierà automaticamente perché condividono la stessa struttura dei WH51 nell'API. L'unico lavoro sarà eventualmente esporre il dato di conducibilità elettrica del substrato (utile per la fertilità) come campo aggiuntivo del EcowittSeriesPoint.

Validazione storica su un anno (piccola, contemplativa)

Usare l'archivio Open-Meteo (1940-oggi) per ricostruire l'evoluzione idrica di un vaso virtuale a Milano per un anno intero, e produrre un grafico stagionale che mostri come il sistema risponde all'inverno, alla primavera, all'estate e all'autunno. Sarebbe un esempio bello di back-testing della libreria, e probabilmente farebbe emergere bias stagionali del modello che oggi non vediamo.

Appendice: comandi utili

Come ripartire dal progetto dopo una pausa lunga.

Setup ambiente

```
$ unzip fitosim.zip
$ cd fitosim/
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install matplotlib numpy pytest
```

Esecuzione test

```
$ python -m pytest tests/                # tutti i test
$ python -m pytest tests/ -v             # verboso
$ python -m pytest tests/test_pot.py     # solo un modulo
```

Esempi rappresentativi da eseguire

```
$ PYTHONPATH=src python examples/multispecies_comparison.py
$ PYTHONPATH=src python examples/inventory_status_report.py
$ PYTHONPATH=src python examples/scheduler_demo.py
$ PYTHONPATH=src python examples/ecowitt_dashboard.py
```

Configurazione credenziali Ecowitt (per esempi reali)

```
$ export ECOWITT_APPLICATION_KEY="..."
```

```
$ export ECOWITT_API_KEY="..."
$ export ECOWITT_MAC="88:13:BF:CB:5A:AF"
```